Introduction to Object-Oriented Programming (Java)

Practice set for weeks 1–2 (Lessons 1–9)

How to use this sheet

There are three exercises of increasing difficulty. Each one consists of

(a) a *specification* to implement;

(b) one or more *test programs* with their expected output. Where the output is given, it must match yours character for character;

(c) *written questions* (Q). Answer each in a few sentences, and justify every claim. "It works when I run it" is not a justification. Answer the questions marked *before coding* before writing any code.

Allowed toolbox. Everything from *Initiation à la programmation en Java* (BOOC I) and Lessons 1–9 of the OOP course. The following come later in the course and are **not** allowed: inheritance (`extends`), interfaces, exceptions (`throw`, `try`), and `static` attributes other than constants (`private static final`). Static methods are only allowed in the class that contains `main`, as in BOOC I. Every attribute is `private`.

Conventions.

- `equals` has the signature seen in Lesson 9: `public boolean equals(C other)`, where `C` is the class itself. It returns `false` when `other` is `null`.
- Because `equals(Object)` has not been covered yet, the `ArrayList` methods `contains`, `indexOf` and `remove(Object)` will *not* call your `equals`. Write the loops yourself.
- Exceptions are not available yet, so invalid operations report failure through their return value (`false`, `null` or `-1`), exactly as specified.

What is tested. • = central, ○ = used.

Concept	Lesson	Ex. 1	Ex. 2	Ex. 3
Classes, attributes, methods, `this`, masking	2–3	○	○	○
`public`/`private`, getters and setters	3	•	○	○
Encapsulation: class invariants, interface design	4–5	•	•	•
Constructors, overloading, chaining with `this(...)`	6–7	•	•	○
Default constructor rules	7	•	○	
Copy constructor; shallow vs. deep copy	8	○	•	•
References, aliasing, `null`, `==`	3, 9	○	•	•
`toString`, `equals`	9	•	•	•
End of life, garbage collection	9		○	○
Arrays, `ArrayList`, `String` (BOOC I)	–	○	•	•

Exercise 1 Exact fractions and the hunt for π

Difficulty: easy. Indicative time: 2 hours.

In Java, `0.1 + 0.2 == 0.3` is `false`, because a `double` cannot store most decimal fractions exactly. In this exercise you build an exact `Fraction` type, then use it to compute harmonic numbers exactly and to find good rational approximations of π.

1.1 The class `Fraction`

A `Fraction` stores its numerator and denominator in two `private final long` attributes. Every `Fraction` object must satisfy the following *class invariant* from the moment its constructor returns:

(I1) the denominator is strictly positive;

(I2) numerator and denominator have no common divisor other than 1 (so zero is stored as $0/1$).

`Fraction` objects are *immutable*: no method ever changes the attributes after construction, and there are no setters.

Constructors

- `Fraction(long n, long d)` builds n/d in normalised form. If `d == 0`, it prints exactly `Error: zero denominator, fraction set to 0` on its own line and builds $0/1$.
- `Fraction(long n)` builds the integer n. *Must delegate* with `this(...)`.
- `Fraction()` builds 0. *Must delegate* with `this(...)`.
- `Fraction(Fraction other)` is the copy constructor.

Methods

`long getNumerator(), long getDenominator()`	accessors.
`Fraction add(Fraction o)`	$this + o$ as a *new* object. Never modifies `this` or `o`.
`Fraction subtract(Fraction o)`	$this - o$.
`Fraction multiply(Fraction o)`	$this \times o$.
`Fraction divide(Fraction o)`	$this/o$; returns `null` if o is zero.
`Fraction reciprocal()`	$1/this$; returns `null` if `this` is zero.
`boolean isLessThan(Fraction o)`	exact comparison $this < o$. No `double` allowed.
`double toDouble()`	approximate value.
`String toString()`	`"n/d"`, or just `"n"` when the denominator is 1.
`boolean equals(Fraction o)`	same rational number.

Write the greatest-common-divisor computation as a `private` helper method (Euclid's algorithm). You may assume that no test overflows `long`.

Test 1

```
class FractionTest {
    public static void main(String[] args) {
        Fraction a = new Fraction(6, -8);
        Fraction b = new Fraction(1, 4);
        System.out.println(a + " " + b + " " + new Fraction(10, 5)
```

```
                        + " " + new Fraction(0, -7) + " " + new Fraction());
        System.out.println(a.add(b) + " " + a.subtract(b) + " "
                        + a.multiply(b) + " " + a.divide(b));
        System.out.println(a.divide(new Fraction()));
        System.out.println(new Fraction(2, 4).equals(new Fraction(-3, -6)));
        System.out.println(a.equals(null) + " " + a.isLessThan(b) + " " + b.isLessThan(a));
        Fraction c = new Fraction(a);
        System.out.println((c == a) + " " + c.equals(a));
        System.out.println(new Fraction(3, 0));
    }
}
```

Expected output:

```
-3/4 1/4 2 0 0
-1/2 -1 -3/16 -3
null
true
false true false
false true
Error: zero denominator, fraction set to 0
0
```

1.2 Harmonic numbers

In a class `FractionLab` (containing `main`), compute and print the harmonic numbers $H_n = 1 + \frac{1}{2} + \cdots + \frac{1}{n}$ exactly for $n = 1, \ldots, 10$, followed by H_{20}:

```
H_1 = 1
H_2 = 3/2
H_3 = 11/6
H_4 = 25/12
H_5 = 137/60
H_6 = 49/20
H_7 = 363/140
H_8 = 761/280
H_9 = 7129/2520
H_10 = 7381/2520
H_20 = 55835135/15519504
```

1.3 Continued fractions

A (simple) continued fraction is written $[a_0; a_1, \ldots, a_k]$ and means

$$[a_0; a_1, \ldots, a_k] \;=\; a_0 + \cfrac{1}{a_1 + \cfrac{1}{\ddots + \cfrac{1}{a_k}}} \qquad (a_0 \in \mathbb{Z},\ a_i \geq 1 \text{ for } i \geq 1).$$

For example $[4; 2, 6, 7] = 415/93$.

(a) In `FractionLab`, write the static function `static Fraction fromContinuedFraction( long[] a)`, which evaluates $[a_0; a_1, \ldots, a_k]$ exactly. You may assume that `a.length >= 1` and that `a[i] >= 1` for `i >= 1`.

(b) In `Fraction`, write the method `ArrayList<Long> toContinuedFraction()`, which returns the *canonical* expansion of `this`. Let $a_0 = \lfloor x \rfloor$. If x is an integer, the expansion is $[a_0]$.

Otherwise it is a_0 followed by the expansion of $1/(x - a_0)$. Careful: Java's integer division truncates toward zero, which is not $\lfloor \cdot \rfloor$ for negative numbers. You may use `Math.floorDiv(long, long)`.

Test 2

```
System.out.println(FractionLab.fromContinuedFraction(new long[]{4, 2, 6, 7}));
System.out.println(FractionLab.fromContinuedFraction(new long[]{3, 7, 15, 1}));
System.out.println(FractionLab.fromContinuedFraction(new long[]{-3, 1, 2}));
System.out.println(FractionLab.fromContinuedFraction(new long[]{5}));
System.out.println(new Fraction(415, 93).toContinuedFraction());
System.out.println(new Fraction(355, 113).toContinuedFraction());
System.out.println(new Fraction(-7, 3).toContinuedFraction());
System.out.println(new Fraction(5).toContinuedFraction());
System.out.println(new Fraction(1, 3).toContinuedFraction());
System.out.println(new Fraction(0).toContinuedFraction());
```

Expected output:

```
415/93
355/113
-7/3
5
[4, 2, 6, 7]
[3, 7, 16]
[-3, 1, 2]
[5]
[0, 3]
[0]
```

(c) **Convergents of π.** We have $\pi = [3; 7, 15, 1, 292, 1, 1, \dots]$. The k-th *convergent* is the fraction given by the first k terms. Print the first seven convergents, each with its decimal value and its absolute error $|c - \pi|$ (use `Math.PI`). The layout is up to you, but the fractions must be the following:

```
3              3.0000000000  error 1.42e-01
22/7           3.1428571429  error 1.26e-03
333/106        3.1415094340  error 8.32e-05
355/113        3.1415929204  error 2.67e-07
103993/33102   3.1415926530  error 5.78e-10
104348/33215   3.1415926539  error 3.32e-10
208341/66317   3.1415926535  error 1.22e-10
```

(d) **Record hunt.** For every denominator $d = 1, \dots, 1000$, let `n = Math.round(Math.PI * d)` and consider n/d. Print each fraction whose error is *strictly* smaller than the error of every fraction seen before. You should find exactly these 14 records:

```
3  13/4  16/5  19/6  22/7  179/57  201/64  223/71  245/78  267/85
289/92  311/99  333/106  355/113
```

1.4 Questions

Q1.1 *(Lessons 6–7.)* List, in execution order, every constructor body that runs during `new Fraction()`. Next, suppose you deleted every constructor except `Fraction(long, long)`. Which expressions in Test 1 would no longer compile? Why does Java not supply a default constructor in that case?

Q1.2 *(Invariant.)* Your `equals` compares attributes one by one. Explain why this is correct *only because of* (I1) and (I2). Give two attribute pairs representing $\frac{1}{2}$ that a non-normalising constructor could produce, and that `equals` would wrongly call different. Then give a pair violating (I1) for which your `isLessThan` gives a wrong answer, and show the computation.

Q1.3 *(Encapsulation and aliasing.)* A classmate removes `final` and adds

```
public void setNumerator(long n) { num = n; }
```

(a) Write a client snippet of two or three lines after which `equals` returns a wrong result. (b) With this setter, what does `Fraction b = a; a.setNumerator(5);` do to `b`? Draw the memory diagram. (c) Why is `Fraction b = a;` harmless in *your* class? Given that, is your copy constructor useful at all?

Q1.4 *(`null`.)* What happens when `a.divide(new Fraction()).add(b)` runs, and on which exact sub-expression? Compare the strategy of `divide` (return `null`) with that of the constructor (print a message and build 0/1). Which failure is easier to notice and debug, and why? Neither is fully satisfactory. Which later chapter of the course addresses this?

Q1.5 *(`==` vs. `equals`.)* Predict and justify the values of `new Fraction(1, 2) == new Fraction (2, 4)`, `new Fraction(1, 2).equals(new Fraction(2, 4))`, and `(c == a)` in Test 1.

Q1.6 *(Continued fractions.)* `fromContinuedFraction` returns 355/113 for both $\{3, 7, 15, 1\}$ and $\{3, 7, 16\}$. Explain why. Does this mean `toContinuedFraction` is wrong? Also, what would your method return for $-7/3$ if it used `/` instead of the floor? Derive it by hand.

Q1.7 *(Optional, mathematical.)* Only four of the 14 records are convergents. Look at the records from 22/7 to 333/106 and compute the successive differences of their numerators and denominators. Explain the pattern using the *mediant* $\frac{a}{b} \oplus \frac{c}{d} = \frac{a+c}{b+d}$, and relate the number of intermediate records to the term 15 in the expansion of π.

Exercise 2 Turtle graphics on an ASCII canvas

Difficulty: medium. Indicative time: 3–4 hours.

A *turtle* is a small robot holding a pen. It walks over a sheet of paper and leaves a trace wherever it goes while the pen is down. You will build the sheet (`Canvas`), the robot (`Turtle`), and an interpreter for a small drawing language, which you will then use to draw spirals and fireworks in the terminal.

Coordinates. Cell (x, y) is in column x (from 0 on the left) and row y (from 0 at the *top*). "North" means *up*, that is, decreasing y. Headings are multiples of 45°, measured clockwise from North:

heading	0	45	90	135	180	225	270	315
direction	N	NE	E	SE	S	SW	W	NW
$(\Delta x, \Delta y)$	$(0,-1)$	$(1,-1)$	$(1,0)$	$(1,1)$	$(0,1)$	$(-1,1)$	$(-1,0)$	$(-1,-1)$

2.1 The class `Canvas`

A canvas is a rectangular grid of characters. Empty cells contain the constant `BLANK = '.'` (a `private static final char`). The grid is stored in a private two-dimensional `char` array that is *never* exposed outside the class.

`Canvas(int width, int height)`	all cells blank. Any dimension smaller than 1 is replaced by 1.
`Canvas()`	a 21×11 canvas. Must delegate with `this(...)`.
`Canvas(Canvas other)`	an independent copy.
`int getWidth(), int getHeight()`	dimensions.
`boolean isInside(int x, int y)`	whether (x, y) is a cell of the canvas.
`boolean plot(int x, int y, char c)`	writes `c` at (x, y). Returns `false` and does nothing if (x, y) is outside.
`char charAt(int x, int y)`	content of (x, y), or `BLANK` if outside.
`void clear()`	blanks every cell.
`int inkCount()`	number of non-blank cells.
`String toString()`	the rows from top to bottom, separated by `'\n'`, with no trailing newline.
`boolean equals(Canvas o)`	same dimensions and same characters.

2.2 The class `Turtle`

A turtle knows the canvas it draws on (a reference), its position, its heading, whether its pen is down, and the character its pen writes.

`Turtle(Canvas canvas, int x, int y)`	heading 0, pen down, pen character `'*'`. Draws nothing.
`Turtle(Canvas canvas)`	starts at the centre `(width/2, height/2)` (integer division). Must delegate with `this(...)`. Must not crash when `canvas` is `null`; it then starts at $(0, 0)$.
`Turtle(Turtle other)`	same state, and *the same canvas* (shared, not copied).
`getX(), getY(), getHeading()`	position, and heading in degrees in $[0, 360)$.
`isPenDown(), penUp(), penDown()`	pen state.
`void setPen(char c)`	pen character.

`boolean forward(int n)`

If `n < 0`, return `false` and do nothing. Otherwise, if the pen is down, first plot the current cell. Then repeat `n` times: move one cell in the direction of the heading, and plot the new cell if the pen is down. Return `true`. The turtle may leave the canvas: its position is still updated, and out-of-canvas cells are simply not drawn. With a `null` canvas the turtle moves but draws nothing. (Note that `forward(0)` draws a single dot.)

`boolean turnRight(int degrees)`, `boolean turnLeft(int degrees)`

`degrees` may be any multiple of 45, including negative values and values ≥ 360. Otherwise, return `false` and do nothing.

`toString()` **and** `equals(Turtle o)`

`toString` returns, for example, `Turtle at (8,1) heading 90 pen down '*'` (or `pen up`). Two turtles are `equals` when they have the same position, heading, pen state and pen character, *and* draw on the same canvas object.

2.3 A tiny drawing language

Add the method `boolean execute(String program)` to `Turtle`. A program is a sequence of tokens separated by one or more spaces:

`F n`	`forward(n)`
`L d  R d`	`turnLeft(d)`, `turnRight(d)`
`U  D`	pen up, pen down
`P c`	`setPen(c)`, where `c` is a single character
`REPEAT k [ ... ]`	execute the body $k \geq 0$ times. The brackets are separate tokens.

In this basic version, a `REPEAT` body never contains another `REPEAT`. Commands run from left to right. If they all succeed, `execute` returns `true`. At the *first* erroneous command it stops and returns `false`; commands already executed remain executed. A command is erroneous if it is unknown, if its argument is missing, if `forward` or a turn returns `false`, if the argument of `P` is not a single character, or if a `REPEAT` has a negative count, no `[`, or no matching `]`. The empty program and a blank program both return `true`. You may assume that numeric arguments are well-formed integers (`Integer.parseInt`). To split the program into tokens, you may use `String[] tokens = program.trim().split(" +");`.

Test 1

```
class TurtleTest {
    public static void main(String[] args) {
        Canvas c = new Canvas(11, 7);
        Turtle t = new Turtle(c);
        System.out.println(t);
        t.forward(2);
```

```
        t.turnRight(90);
        t.forward(3);
        System.out.println(t);
        System.out.println(c);
        System.out.println("ink: " + c.inkCount());

        Canvas sq = new Canvas(9, 6);
        Turtle s = new Turtle(sq, 1, 4);
        System.out.println(s.execute("REPEAT 4 [ F 3 R 90 ]"));
        System.out.println(sq);
        System.out.println(s);

        Canvas dia = new Canvas(9, 9);
        Turtle d = new Turtle(dia, 4, 0);
        d.setPen('#');
        System.out.println(d.execute("R 135   REPEAT 4 [ F 4 R 90 ]"));
        System.out.println(dia);

        Canvas small = new Canvas(5, 5);
        Turtle o = new Turtle(small, 2, 2);
        System.out.println(o.execute("F 4 R 180 F 6"));
        System.out.println(small);
        System.out.println(o);
        System.out.println(o.execute("F 2 X 3") + " " + o);
        System.out.println(o.execute("R 30") + " " + o.execute("F -2") + " "
                           + o.execute("REPEAT 3 [ F 1") + " " + o.execute(""));
        System.out.println(o.execute("L 450") + " " + o);
    }
}
```

Expected output (shown in two columns: read the left column first):

```
Turtle at (5,3) heading 0 pen down '*'
Turtle at (8,1) heading 90 pen down '*'
...........
.....****..
.....*.....
.....*.....
...........
...........
...........
ink: 6
true
.........
.****....
.*..*....
.*..*....
.****....
.........
Turtle at (1,4) heading 0 pen down '*'
```

```
true
....#....
...#.#...
..#...#..
.#.....#.
#.......#
.#.....#.
..#...#..
...#.#...
....#....
true
..*..
..*..
..*..
..*..
..*..
Turtle at (2,4) heading 180 pen down '*'
false Turtle at (2,6) heading 180 pen down '*'
false false false true
true Turtle at (2,6) heading 90 pen down '*'
```

Test 2: a spiral driven from Java, and two turtles sharing one canvas

```
Canvas sp = new Canvas(13, 13);
Turtle p = new Turtle(sp);
for (int i = 1; i <= 12; ++i) {
    p.forward(i);
    p.turnRight(90);
}
System.out.println(sp);
```

```
System.out.println(p + ", ink: " + sp.inkCount());

Canvas shared = new Canvas(15, 5);
Turtle t1 = new Turtle(shared, 0, 1);
Turtle t2 = new Turtle(t1);
System.out.println(t1.equals(t2));
t1.execute("R 90 F 14");
t2.execute("P o R 135 F 3 L 45 F 11");
System.out.println(t1.equals(t2));
System.out.println(shared);
```

```
.............
..***********
..*.........*
..*.*******.*
..*.*.....*.*
..*.*.***.*.*
..*.*.*.*.*.*
..*.*...*.*.*
..*.*****.*.*
..*.......*.*
..*********.*
............*
*************
Turtle at (0,12) heading 0 pen down '*', ink: 79
```

```
true
false
...............
o**************
.o.............
..o............
...ooooooooooo
```

2.4 Animation frames (before running)

The following code records the frames of a small animation in two different ways. *Before running it*, write down its complete output by hand. Then run it and compare. Question Q2.4 refers to this code.

```
Canvas f = new Canvas(5, 3);
Turtle ft = new Turtle(f, 0, 1);
ft.turnRight(90);
ArrayList<Canvas> good = new ArrayList<Canvas>();
ArrayList<Canvas> bad = new ArrayList<Canvas>();
for (int i = 0; i < 3; ++i) {
    ft.forward(1);
    good.add(new Canvas(f));
    bad.add(f);
}
for (int i = 0; i < 3; ++i) {
    System.out.println(good.get(i));
    System.out.println(bad.get(i));
    System.out.println();
}
System.out.println(good.get(2).equals(bad.get(0)) + " " + (good.get(2) == bad.get(0))
                   + " " + (bad.get(0) == bad.get(2)));
```

2.5 Puzzle: the firework

Starting from `new Turtle(new Canvas(11, 11))`, write a program for `execute` with **at most 25 tokens** that produces exactly this picture (the centre cell holds an `o`):

```
*....*....*
.*...*...*.
..*..*..*..
...*.*.*...
...........
****.o.****
...........
...*.*.*...
..*..*..*..
.*...*...*.
*....*....*
```

2.6 Bonus: nested loops

Extend `execute` so that `REPEAT` bodies may contain other `REPEAT`s, to any depth. Brackets must then be matched properly. With `new Turtle(new Canvas(19, 5), 1, 3)`, the program `REPEAT 3 [ REPEAT 4 [ F 2 R 90 ] U R 90 F 4 L 90 D ]` must return `true` and draw:

```
...................
.***.***.***.......
.*.*.*.*.*.*.......
.***.***.***.......
...................
```

2.7 Questions

Q2.1 *(Copy semantics.)* The copy constructor of `Canvas` copies the grid, but the copy constructor of `Turtle` shares the canvas. Justify *both* choices from what each object represents. Draw the memory diagram right after `Turtle t2 = new Turtle(t1);` in Test 2. What would Test 2 print if `Turtle(Turtle)` copied the canvas instead?

Q2.2 *(Identity vs. equality.)* `Turtle.equals` compares canvases with `==`, not with `Canvas.equals`. Describe a situation where the two tests disagree. Argue which one matches the meaning "these two turtles are interchangeable".

Q2.3 *(Constructors.)* Explain precisely why the naive delegation

```
this(canvas, canvas.getWidth() / 2, canvas.getHeight() / 2);
```

crashes for a `null` canvas. Why can you not simply put an `if` before `this(...)`? How did you solve it? Finally, why does the call `new Turtle(null)` not even compile, so that one must write `new Turtle((Canvas) null)`?

Q2.4 *(References and end of life.)* For the code of 2.4: how many `Canvas` objects exist after the loop? Suppose that after the loop we execute `f = null; bad = null;`. Which `Canvas` objects become eligible for garbage collection? Be careful: count every reference that still points to each object.

Q2.5 *(Encapsulation.)* Suppose `Canvas` offered

```
public char[][] getCells() { return cells; }
```

Write two client lines that break a property every other method relies on, and name a method that would then crash. Relate this to the `JeuMorpion.getGrille()` example of Lesson 5.

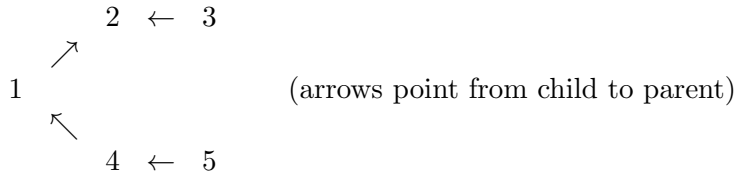
Q2.6 *(Edge cases.)* What is `-1 % 8` in Java? Show step by step how your heading update handles `turnLeft(450)` from heading 180, and `turnRight(-45)` from heading 0.

Exercise 3 MiniGit: a tiny version-control system

Difficulty: hard. Indicative time: 5–6 hours.

Version-control systems such as `git` record *snapshots* of your files over time. In this exercise you build a miniature one for a single text document. Every bug in it comes from an object that should have been copied but was shared, or should have been shared but was copied, so this exercise is a thorough test of Lessons 8 and 9.

Model. A `Repository` owns a *working copy* (the `Document` that the user edits) and a history of `Commits`. A commit is a frozen snapshot of the document, with an id, a message and a *parent* commit (`null` for the very first commit). The repository also knows its *head*, the commit the working copy was last synchronised with. After a checkout of an old commit, new commits start a new branch, so the history is a tree. In the test below it looks like this:



Warm-up (before coding). Suppose the `Commit` constructor were written with `this.snapshot = content;` instead of making a copy. Everything else is correct. What would the following code print? Answer with a memory diagram (this is Q3.1).

```
Repository repo = new Repository();
Document doc = repo.getWorkingCopy();
doc.appendLine("v1");
System.out.println(repo.commit("first"));
doc.setLine(0, "v2");
System.out.println(repo.commit("second"));
System.out.println(repo.log());
```

3.1 The class `Document` (mutable)

A document is a list of lines, stored in a `private ArrayList<String>`. No line is ever `null`.

`Document()`	empty document.
`Document(String[] lines)`	a document with these lines. The array itself must not be kept.
`Document(Document other)`	an independent copy.
`int lineCount()`	number of lines.
`String getLine(int i)`	line `i`, or `null` if `i` is invalid.
`boolean appendLine(String s)`	adds `s` at the end.
`boolean insertLine(int i, String s)`	inserts `s` before line `i`, where $0 \le i \le$ `lineCount()`.
`boolean setLine(int i, String s)`	replaces line `i`.
`boolean removeLine(int i)`	removes line `i`.
`void replaceContentWith(Document other)`	makes `this` contain a copy of the lines of `other`. Does nothing if `other` is `null`. Must also work when `other == this`.
`String toString()`	each line followed by `'\n'`(empty document: `""`).
`boolean equals(Document o)`	same lines in the same order.

All modifiers return `false` and change nothing when the index is invalid or the line is `null`. Otherwise they return `true`.

3.2 The class `Commit` (immutable)

`Commit(int id, String message, Document content, Commit parent)`	stores an *independent* copy of `content`. Later changes to `content` must not affect the commit.
`getId(), getMessage(), getParent()`	accessors.
`Document getSnapshot()`	a *new independent copy* of the stored document.
`boolean hasSameContentAs(Document d)`	compares with `d` *without* making a copy.
`int depth()`	number of ancestors (the root has depth 0).
`boolean isAncestorOf(Commit other)`	`true` iff this very object is `other` or one of its ancestors.
`String toString()`	`[3] Serving suggestion (parent 2)` or `[1] First version (root)`.

3.3 The class `Repository`

A repository has a working document, a head (`null` before the first commit), the list of all its commits, and a counter for the next id. Ids are $1, 2, 3, \ldots$ in creation order. They are assigned by the repository itself (no `static` attribute).

`Repository()`	empty working document, no commit.
`Document getWorkingCopy()`	the working document *itself* (not a copy), so that clients can edit it. Throughout the life of the repository, it always returns the *same object*.
`int getHeadId()`	id of the head, or -1.
`boolean hasUncommittedChanges()`	without a head: whether the working copy is non-empty. Otherwise: whether it differs from the head's snapshot.
`int commit(String message)`	if there is nothing to commit, returns -1 and does nothing. Otherwise creates a commit (next id, parent = current head), makes it the head and returns its id.
`boolean checkout(int id)`	returns `false` and does nothing if `id` is unknown or if there are uncommitted changes. Otherwise the commit becomes the head and the working copy receives its content, *in place*.
`String log()`	the head, then its parent, and so on up to the root, one `toString()` per line, separated by `'\n'`, no trailing newline. Without commits: `(no commits)`.
`int mergeBase(int a, int b)`	id of the *deepest* commit that is an ancestor of both (every commit is its own ancestor); -1 if an id is unknown.
`String diff(int a, int b)`	line diff from snapshot `a` to snapshot `b` (below); `null` if an id is unknown.

The diff algorithm

Let $a_0, \ldots, a_{n-1}$ be the lines of the first snapshot and $b_0, \ldots, b_{m-1}$ the lines of the second one.

Define $L[i][j]$ as the length of a longest common subsequence of $a_i, \ldots, a_{n-1}$ and $b_j, \ldots, b_{m-1}$:

$$L[n][j] = L[i][m] = 0, \qquad L[i][j] = \begin{cases} 1 + L[i{+}1][j{+}1] & \text{if } a_i \text{ equals } b_j, \\ \max\big(L[i{+}1][j],\, L[i][j{+}1]\big) & \text{otherwise.} \end{cases}$$

Fill the table in a `int[n+1][m+1]` array. Then start with $i = j = 0$ and repeat while $i < n$ or $j < m$:

1. if $i < n$, $j < m$ and a_i equals b_j: output `"  "` $+\, a_i$ (two spaces), then increment i and j;
2. else if $j = m$, or ($i < n$ and $L[i{+}1][j] \geq L[i][j{+}1]$): output `"- "` $+\, a_i$, then increment i;
3. else output `"+ "` $+\, b_j$, then increment j.

Each output line ends with `'\n'`. The tie-break in rule 2 (removals before additions) is part of the specification: it makes the output unique.

3.4 Test

```
class MiniGitTest {
    public static void main(String[] args) {
        Repository repo = new Repository();
        Document doc = repo.getWorkingCopy();
        System.out.println(repo.log());
        System.out.println(repo.commit("Empty"));

        doc.appendLine("PANCAKES");
        doc.appendLine("200 g flour");
        doc.appendLine("2 eggs");
        doc.appendLine("300 ml milk");
        System.out.println(repo.commit("First version"));
        System.out.println(repo.commit("Nothing new"));

        doc.insertLine(3, "1 pinch of salt");
        doc.setLine(4, "350 ml milk");
        System.out.println(repo.commit("Salt, more milk"));
        doc.appendLine("Serve with maple syrup");
        System.out.println(repo.commit("Serving suggestion"));
        System.out.println(repo.log());

        doc.removeLine(0);
        System.out.println(repo.checkout(1));             // refused
        doc.insertLine(0, "PAN" + "CAKES".substring(0));  // built at run time
        System.out.println(repo.hasUncommittedChanges());
        System.out.println(repo.checkout(1));
        System.out.print(doc);
        System.out.println(doc == repo.getWorkingCopy());

        doc.setLine(0, "VEGAN PANCAKES");
        doc.setLine(2, "1 ripe banana");
        doc.setLine(3, "300 ml oat milk");
        System.out.println(repo.commit("Vegan variant"));
        doc.appendLine("Serve with berries");
        System.out.println(repo.commit("Berries"));
        System.out.println(repo.log());

        System.out.println(repo.mergeBase(3, 5) + " " + repo.mergeBase(2, 3) + " "
                           + repo.mergeBase(4, 4) + " " + repo.mergeBase(1, 9));
        System.out.print(repo.diff(1, 2));
        System.out.println("--");
        System.out.print(repo.diff(3, 5));
```

```
        System.out.println("--");
        System.out.println(repo.diff(2, 7));
        System.out.println(repo.checkout(3) + " " + doc.lineCount() + " "
                           + doc.getLine(5) + " " + doc.getLine(6));
    }
}
```

Expected output (two columns):

```
(no commits)
-1
1
-1
2
3
[3] Serving suggestion (parent 2)
[2] Salt, more milk (parent 1)
[1] First version (root)
false
false
true
PANCAKES
200 g flour
2 eggs
300 ml milk
true
4
5
[5] Berries (parent 4)
[4] Vegan variant (parent 1)
[1] First version (root)
```

```
1 2 4 -1
  PANCAKES
  200 g flour
  2 eggs
- 300 ml milk
+ 1 pinch of salt
+ 350 ml milk
--
- PANCAKES
+ VEGAN PANCAKES
  200 g flour
- 2 eggs
- 1 pinch of salt
- 350 ml milk
- Serve with maple syrup
+ 1 ripe banana
+ 300 ml oat milk
+ Serve with berries
--
null
true 6 Serve with maple syrup null
```

3.5 Play with it: an interactive shell

Write a class `MiniGitShell` whose `main` reads commands from the keyboard (`Scanner`) and acts on a `Repository`, so that you can version your own texts:

```
show, append <text>, insert <i> <text>, set <i> <text>, delete <i>,
  status, commit <message>, log, checkout <id>, diff <a> <b>, quit
```

The exact messages are up to you. The shell must never crash on an invalid index or id: it reports the failure instead. You may assume that numeric arguments are well-formed integers.

3.6 Questions

Q3.1 *(Warm-up, before coding.)* Answer the warm-up above. Then run it on your correct implementation and explain the difference.

Q3.2 *(Interface design.)* `getWorkingCopy` returns an internal reference, while `getSnapshot` returns a copy. Is this inconsistent? Justify both choices: which object is *meant* to be modified by clients, and which property of the system would a leaking `getSnapshot` destroy? Write a three-line client snippet that demonstrates the damage.

Q3.3 *(In-place update.)* What exactly does the line `doc == repo.getWorkingCopy()` of the test check? Describe the scenario in which a user silently loses edits if `checkout` did `working = c.getSnapshot();` instead.

Q3.4 *(Safe sharing.)* `getParent()` returns an internal reference without copying it. Is this an encapsulation leak like the one in Q3.2? Commit 1 is shared by two branches. Why is this sharing not a problem, while sharing a `Document` would be?

Q3.5 *(Identity vs. equality.)* Why must `isAncestorOf` and `mergeBase` compare commits with `==`? Construct two distinct commits that an attribute-by-attribute `equals` would declare equal, and explain why `mergeBase` could then return a wrong answer.

Q3.6 *(Strings.)* Why does the test build a line with `"PAN" + "CAKES".substring(0)`? Which lines of the expected output would change if `Document.equals` compared lines with `==`, and how?

Q3.7 *(Depth of a copy.)* `Document(Document)` copies the `ArrayList` but shares the `String` objects. Why is this deep enough? What would have to change if lines were stored as mutable objects (for example `StringBuilder`)?

Q3.8 *(Self-assignment.)* Trace `doc.replaceContentWith(doc)` on an implementation that first clears its list and then copies the lines of `other`. What is lost? Explain how your version avoids this.

Q3.9 *(Cost.)* With c commits of a document of about ℓ lines, how much memory do the snapshots use? What are the time and memory costs of `diff` on documents of n and m lines? Propose (without coding it) a representation of commits that uses less memory, and say which public methods would become slower. Explain why the public interface of `Repository` would not need to change.

3.7 Going further (open-ended, ungraded)

- **Named branches.** A `Branch` has a name and a head commit that moves forward on each commit. Explain why `Branch` must be mutable while `Commit` is immutable, and what that implies for sharing and copying.
- **Three-way merge.** Given two heads A and B and their base M = `mergeBase(A, B)`, combine the changes $M \to A$ and $M \to B$. Report a *conflict* when both modify the same line of M.
- **Blame.** For each line of the head's snapshot, find the id of the commit that introduced it.